WEEK 1 · DAY 3 · OVERVIEW

# Debugging




Created by **Jeff Lenamon**

ABOUT 65 MIN WITH EXERCISES / 9 SECTIONS / 8 EXERCISES

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

# The Desk Lead's Fifth Note

A new analyst wrote this week's code for our book. Half of it stops in red. Worse, some of it runs and prints numbers I do not trust: QVNE's income came out at 7,875,000. Fix every cell, find the wrong numbers, and make the book check its own totals.

ABOUT 65 MIN WITH EXERCISES / 9 SECTIONS / 8 EXERCISES

# Your Route Through Day 3

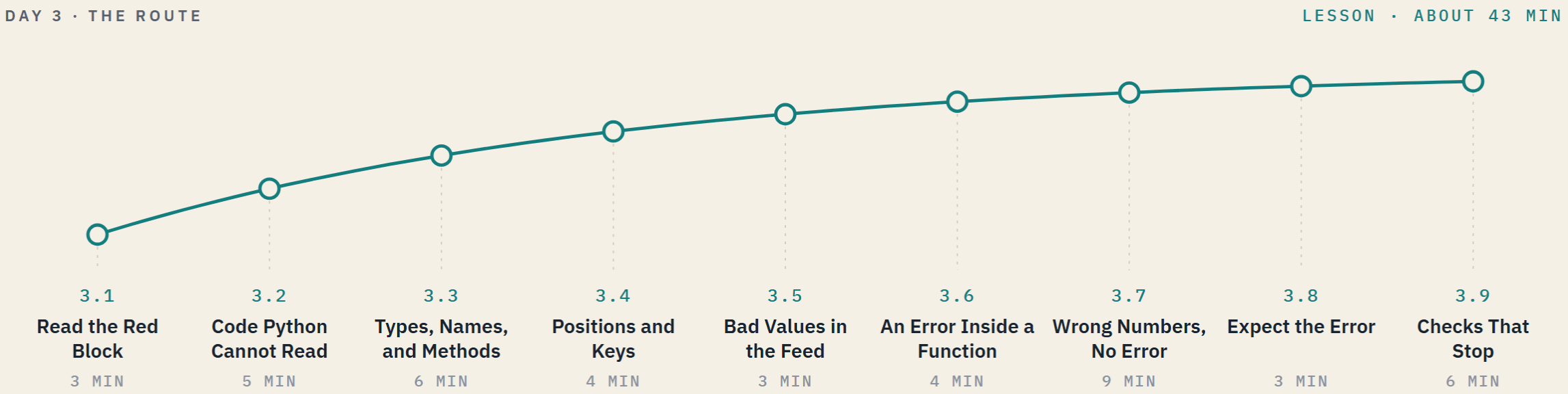



FIGURE 1 Nine sections, in order, with about how long each takes.

**Time. About 43 minutes for the lesson, then about 24 for the eight exercises.**

# 3.1 Read the Red Block

**Read a traceback from the bottom up** — read 1, then 2, then 3

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[2], line 3
      1 # the analyst's first cell: this cell is meant to fail
      2 coupon = 5.25
----> 3 print('The coupon is', cupon)

NameError: name 'cupon' is not defined
```

3 — the cell's run number, and the line in it

2 — the line Python stopped on

1 — the error, and its message

FIGURE 3 The cupon traceback: 1 the last line names the error, 2 the arrow marks the line, 3 the cell and line number.

THE ANALYST'S CELL

```
coupon = 5.25
print('The coupon is', cupon)
```

```
NameError: name 'cupon' is not defined
```

SPELLED AS STORED

```
coupon = 5.25
print('The coupon is', coupon)
```

```
The coupon is 5.25
```

**Every traceback answers three questions: what type, what message, which line.**

# 3.2 Code Python Cannot Read

### A MISSING COLON

```
print('The tickers:')
for ticker in ticker_list
    print(ticker)
```

```
SyntaxError: expected ':'
```

### A BRACKET LEFT OPEN

```
new_coupon_list = [5.25, 6.125, 4.75, 7.5, 6.0
print(new_coupon_list)
```

```
SyntaxError: '[' was never closed
```

### ONE EQUALS SIGN, THEN TWO

```
price = 98.5
if price = 100:
    print('QVNE trades at par')
else:
    print('QVNE does not trade at par')
```

```
SyntaxError: invalid syntax. Maybe you meant '==' or ':=' instead of '='?
```

**From here the code is good Python: it runs until it reaches a line it cannot carry out.**

# 3.3 Types, Names, and Methods

TEXT TIMES A NUMBER

```
face = '1500000'
coupon = 5.25
income = face * coupon / 100
```

```
TypeError: can't multiply sequence by non-int of type 'float'
```

EXCEL SIDE BY SIDE

**Excel side by side.** If A1 holds 1500000 stored as text, `=A1*5.25/100` still returns 78750: Excel's arithmetic converts text that looks like a number.

THE TYPE IS WRONG

```
print(type(face))
print(type(coupon))
print(face * 2)
```

```
<class 'str'>
<class 'float'>
15000001500000
```

CONVERT IT FIRST

```
face = int(face)
income = face * coupon / 100
print('The annual coupon income is', income)
```

```
The annual coupon income is 78750.0
```

**A type error: the operation is fine, the type is wrong.**

# 3.4 Positions and Keys

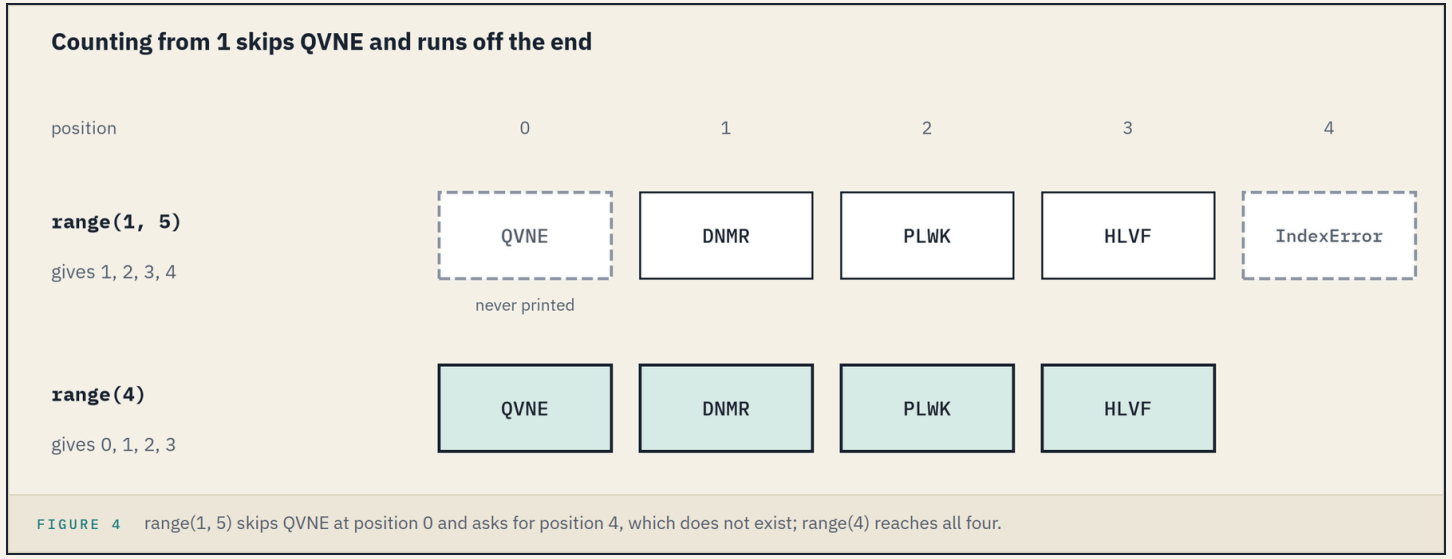



FIGURE 4 range(1, 5) skips QVNE at position 0 and asks for position 4, which does not exist; range(4) reaches all four.

COUNTING FROM ONE

```
for i in range(1, 5):
    print(ticker_list[i])
```

```
IndexError: list index out of range
```

EXCEL SIDE BY SIDE

**Excel side by side.** Excel counts from 1: `=INDEX(A1:A4,1)` returns the first item.

**Counting from 1 is an off-by-one bug, one of the commonest there is.**

# 3.5 Bad Values in the Feed

### A PERCENT SIGN

```
coupon_text = '5.25%'
coupon = float(coupon_text)
```

```
ValueError: could not convert string to float: '5.25%'
```

### EXCEL SIDE BY SIDE

**Excel side by side.** `=VALUE("5.25%")` returns 0.0525: Excel reads the percent sign and divides by 100.

### REPORT THE BAD PRICE AND SKIP IT

```
    if price <= 0:
        print(ticker, 'has a bad price:', price)
    else:
        print(ticker, round(coupon / price * 100, 2))
```

```
QVNE 5.33
DNMR 5.93
PLWK has a bad price: 0
HLVF 7.43
```

**Fix the data, not the formula.**

# 3.6 An Error Inside a Function

**An error inside a function: read the arrows bottom up**

```
Cell In[29], line 4
      ...
----> 4     print(ticker, round(current_yield(coupon, price), 2))
```

**upper arrow**

how it got there: the call

price = '95.0'

```
Cell In[28], line 4, in current_yield(coupon, price)
      ...
----> 4     return coupon / price * 100
```

**lower arrow**

where it failed

```
TypeError: unsupported operand type(s) for /: 'float' and 'str'
```

**read first**

FIGURE 5 A traceback with two frames: the upper arrow on the loop's call with '95.0', the lower on the line that failed.

PART 2'S FUNCTION

```
def current_yield(coupon, price):
    return coupon / price * 100
```

A PRICE THAT CAME THROUGH AS TEXT

```
feed_price_list = [98.5, 103.25, '95.0', 101.0]
```

**Lower arrow: where it failed. Upper arrow: the call that brought the bad value in.**

# Every Error So Far

| ERROR | MEANS | FIRST CHECK | EXCEL'S NEAREST |
|---|---|---|---|
| `SyntaxError`, `IndentationError` | cannot read the code | colon, bracket, quote, indent | formula refused |
| `NameError` | no such name | spelling, quotes, `import`, a cell not run | `#NAME?` |
| `TypeError` | wrong type for the operation | `type()` of each value, as with `'QVNE ' + 5.25` in Part 1 | `#VALUE!` |
| `ValueError` | right type, wrong content | the text converted, such as '5.25%' | `#VALUE!` |
| `IndexError` | no item at that position | the list's length; a `range` one off | `#REF!` |
| `KeyError` | no such key | `.keys()`: spelling and capitals | `#N/A` |
| `AttributeError` | no such attribute on that object | `dir()`, `help()` | `#NAME?` |
| `ZeroDivisionError` | division by 0 | the data | `#DIV/0!` |
| `ModuleNotFoundError` (Day 0) | library not installed | spelling, then `%pip install` | no match |
| `FileNotFoundError` (Day 2) | nothing at that path | file name, then folder | no match |

**Every error so far stopped the run. The worst bugs do not.**

# 3.7 Wrong Numbers, No Error

**An average must land between the lowest and the highest coupon**

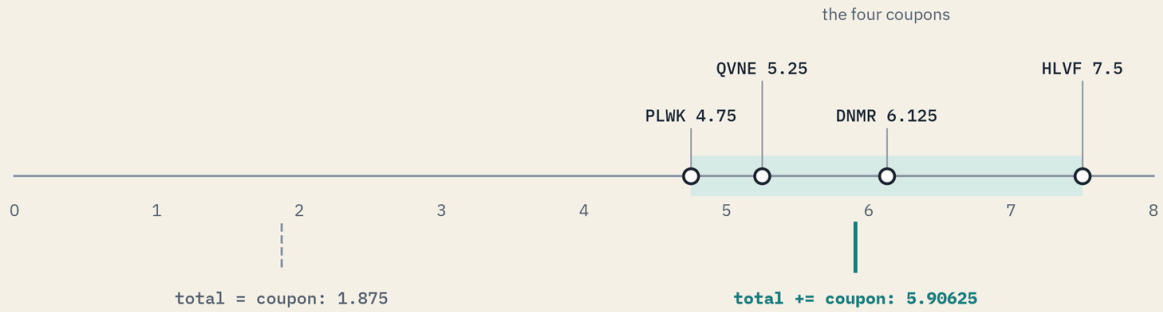



FIGURE 6 The coupons run from 4.75 to 7.5; the wrong average, 1.875, falls outside them, the right one, 5.90625, inside.

INCOME WITHOUT THE PERCENT

```
print(f'As the analyst wrote it: {face * coupon:,.2f}')
```

```
As the analyst wrote it: 7,875,000.00
Coupon divided by 100:   78,750.00
```

AN AVERAGE BELOW THE LOWEST COUPON

```
total = 0
for coupon in coupon_list:
    total = coupon
average_coupon = total / len(coupon_list)
print('The average coupon is', average_coupon)
```

**Know roughly the answer before you run the code.**

# Two Names, One List

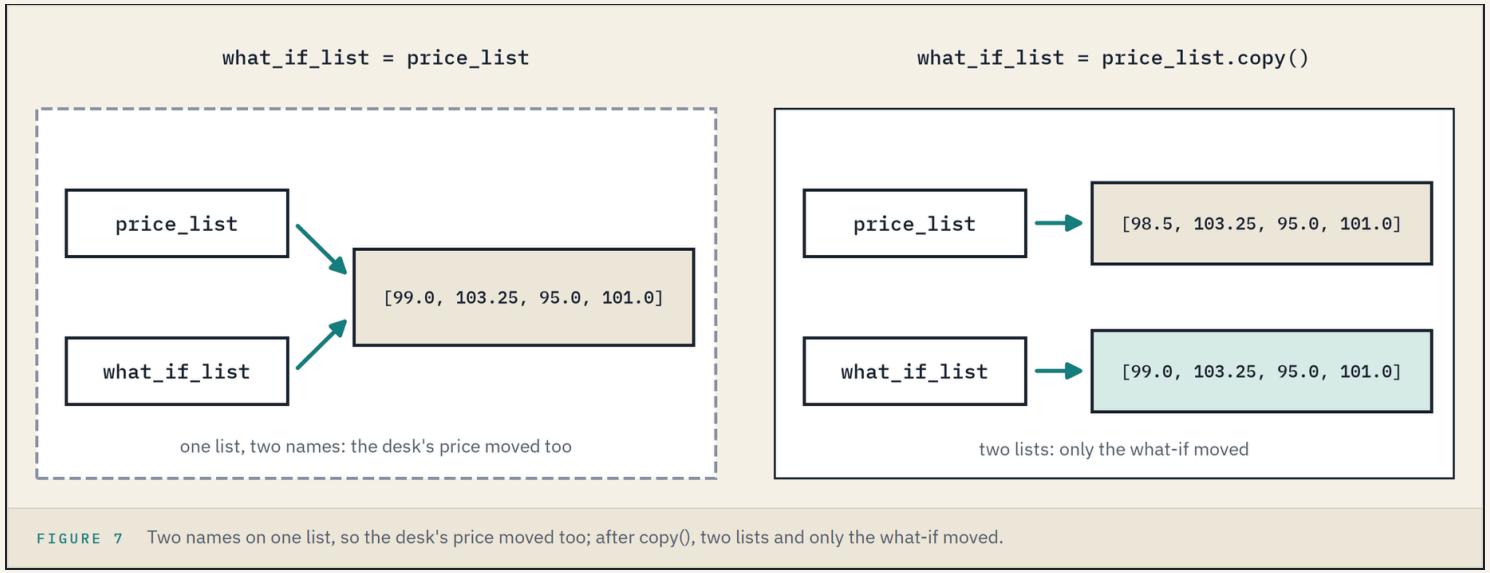



FIGURE 7 Two names on one list, so the desk's price moved too; after copy(), two lists and only the what-if moved.

### A SECOND NAME, NOT A SECOND LIST

```
what_if_list = price_list
what_if_list[0] = 99.0
print(f'What-if prices: {what_if_list}')
print(f'Desk prices:    {price_list}')
```

```
What-if prices: [99.0, 103.25, 95.0, 101.0]
Desk prices:    [99.0, 103.25, 95.0, 101.0]
```

### EXCEL SIDE BY SIDE

**Excel side by side.** A pasted range is a separate copy from the moment it lands; in Python, ask for one with `.copy()`.

**`what_if_list = price_list` made a second name, not a second list, so the 99.0 landed under both (mutable, Part 1).**

# 3.8 Expect the Error

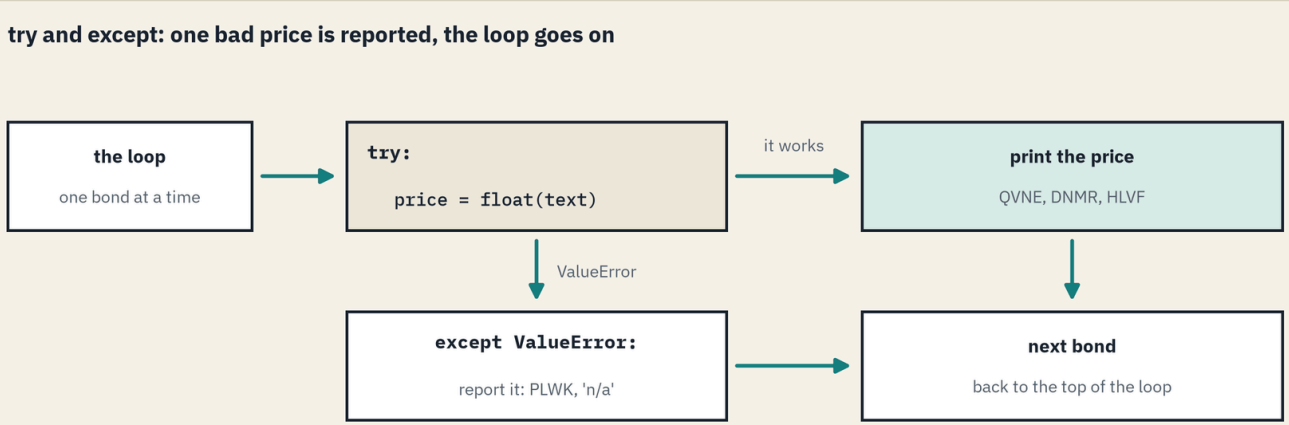



FIGURE 8 Through try, a good price is printed and n/a takes the except ValueError path; both go on to the next bond.

THE SAME LOOP, UNDER TRY

```
    try:
        price = float(text)
    except ValueError:
```

```
QVNE is priced at 98.5
DNMR is priced at 103.25
PLWK has a price that is not a number: n/a
HLVF is priced at 101.0
```

**Always name the error you expect.**

# 3.9 Checks That Stop

### A REPEAT KEYED IN

```
faces_with_repeat = face_list + [250000]
total_face = sum(faces_with_repeat)
```

```
AssertionError: expected 3,000,000 face and found 3,250,000
```

### THE BOOK TIES

```
assert book_face == 3000000, 'face does not tie'
assert math.isclose(book_cost, 2958750), 'cost does not tie'
assert math.isclose(book_income, 163750), 'income does not tie'
```

```
Book ties: 3,000,000 face, $2,958,750.00 cost, $163,750.00 income
```

### EXCEL SIDE BY SIDE

| PYTHON | WHO USES IT | EXCEL'S NEAREST |
|---|---|---|
| `assert` | you, on your own totals | a check cell, `=IF(SUM(F2:F5)=3000000,"OK","CHECK")`, which calculates on |
| `raise` | a function, on a bad value it is handed | Data Validation (Data tab), which refuses a value as it is typed |
| `try` and `except` | the caller, deciding what to do | `IFNA`, as in 3.8 |

**The rule from here on: a check that matters is an `assert` or a `raise`, never a printed number.**

# A Function Refuses, the Caller Decides

A FUNCTION REFUSES

```
    if price <= 0:
        raise ValueError(f'the price must be above zero, and it is {price}')
```

THE CALLER DECIDES

```
    try:
        print(f'{ticker}: {current_yield(coupon, price):.2f} percent')
    except ValueError as problem:
        print(f'{ticker}: skipped, {problem}')
```

```
QVNE: 5.33 percent
DNMR: skipped, the price must be above zero, and it is 0
PLWK: 5.00 percent
HLVF: 7.43 percent
```

**Two arrows again, as in 3.6.**

# You Can Now

- Read a traceback: the error type, the message, and the line.
- Name the common errors and say what each one is telling you.
- Follow an error inside a function back to the line that caused it.
- Catch a wrong number that raises no error.
- Stop a run with **`assert`**, and handle bad data with **`try`** and **`except`**.

| THE NOTE ASKED | ANSWER |
|---|---|
| Fix every cell | 18 red blocks read from the last line up: 16 bugs fixed, and 2 checks doing their job |
| Find the wrong numbers | QVNE's 7,875,000 is 78,750 dollars; the 1.875 average coupon is 5.90625 |
| Make the book check its own totals | three **`assert`** lines tie the face, the cost, and the income |

NEXT
WEEK 1 · DAY 4

## Classes: A Bond Object

Bundling a bond's data and its calculations into one object.

